Neural Networks 171 (2024) 332–342

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

Bio-inspired affordance learning for 6-DoF robotic grasping: A transformer-based global feature encoding approach

Zhenjie Zhao [a,b,c,1], Hang Yu [b,c,1], Hang Wu [b,c,*], Xuebo Zhang [b,c,*]

[a] College of Computer Science, Nanjing University of Information Science and Technology, Nanjing, Jiangsu, China
[b] Institute of Robotics and Automatic Information System, College of Artificial Intelligence, Nankai University, Tianjin, China
[c] Tianjin Key Laboratory of Intelligent Robotics, Nankai University, Tianjin, China

ARTICLE INFO

Keywords:
Affordance learning
Bio-inspired neural network
Transformer
6-DoF robotic grasping

ABSTRACT

The 6-Degree-of-Freedom (6-DoF) robotic grasping is a fundamental task in robot manipulation, aimed at detecting graspable points and corresponding parameters in a 3D space, *i.e* affordance learning, and then a robot executes grasp actions with the detected affordances. Existing research works on affordance learning predominantly focus on learning local features directly for each grid in a voxel scene or each point in a point cloud scene, subsequently filtering the most promising candidate for execution. Contrarily, cognitive models of grasping highlight the significance of global descriptors, such as size, shape, and orientation, in grasping. These global descriptors indicate a grasp path closely tied to actions. Inspired by this, we propose a novel bio-inspired neural network that explicitly incorporates global feature encoding. In particular, our method utilizes a Truncated Signed Distance Function (TSDF) as input, and employs the recently proposed Transformer model to encode the global features of a scene directly. With the effective global representation, we then use deconvolution modules to decode multiple local features to generate graspable candidates. In addition, to integrate global and local features, we propose using a skip-connection module to merge lower-layer global features with higher-layer local features. Our approach, when tested on a recently proposed pile and packed grasping dataset for a decluttering task, surpassed state-of-the-art local feature learning methods by approximately 5% in terms of success and declutter rates. We also evaluated its running time and generalization ability, further demonstrating its superiority. We deployed our model on a Franka Panda robot arm, with real-world results aligning well with simulation data. This underscores our approach's effectiveness for generalization and real-world applications.

1. Introduction

Robotic grasping (Newbury et al., 2022; Platt, 2023) aims to detect graspable points of an object and recognizes their corresponding parameter configurations of a gripper, and then we can select a promising grasp candidate to execute. It is a fundamental skill for robot manipulation, which has such broad applications as clutter removal (Huang, Han, Boularias, & Yu, 2021), surgical assistance (Di Natali, Buzzi, Garbin, Beccani, & Valdastri, 2015), intelligent logistics (Song & Xin, 2021), manufacturing (Oliver, Gil, & Torres, 2020), and service robots (Cong et al., 2021).

The procedure of detecting grasp parameters is called affordance learning (Detry et al., 2011), which is critical for a successful grasp action. State-of-the-art methods of affordance learning formulate it as a machine learning problem (Wu et al., 2020), and use end-to-end trained deep neural networks to predict graspable points and their corresponding gripper parameters from visual observations (Alliegro, Rudorfer, Frattin, Leonardis, & Tommasi, 2022; Breyer et al., 2021). In general, the backbone models are selected according to the input data types. For instance, the PointNet (Qi, Su, Mo, & Guibas, 2017) and PointNet++ (Qi, Yi, Su, & Guibas, 2017) are commonly used for unstructured point cloud (Fang, Wang, Gou, & Lu, 2020; Wu et al., 2020), and 3D convolutional neural networks (3D-CNN) (Kopuklu, Kose, Gunduz, & Rigoll, 2019) is used for structured voxelized space (Breyer et al., 2021).

Cognitive models of grasping, such as the FARS model (Fagg & Arbib, 1998) and the TRoPICALS model (Caligiore, Borghi, Parisi, & Baldassarre, 2010), have demonstrated that primates' brains, including humans, extract global features (e.g., shapes, sizes, orientations)

* Corresponding authors.
E-mail addresses: wuhang1991@nankai.edu.cn (H. Wu), zhangxuebo@nankai.edu.cn (X. Zhang).
[1] Contributed equally.

https://doi.org/10.1016/j.neunet.2023.12.005
Received 19 September 2023; Received in revised form 20 November 2023; Accepted 4 December 2023
Available online 8 December 2023

from objects and integrate them with local visual features to generate graspable candidates. However, conventional backbone models for affordance learning usually encode features locally and rely on such operators as pooling and convolution to aggregate local features layer by layer in order to obtain a global representation. However, the downsampling of pooling operator can result in the loss of global spatial information. Additionally, convolutional layers have a limited receptive field, which means that they are unable to directly encode global information in a single operation. When multiple layers of convolution are used, the resulting global receptive field is obtained indirectly, which can be a slow and inefficient approach for learning global features. Given the significance of global information in grasping (Wang, Zhou, & Kan, 2022), there is a need for designing models that can encode global features more efficiently.

The Transformer model, originally proposed for language modeling (Vaswani et al., 2017), offers a solution by leveraging attention mechanisms to effectively learn both global and local information. In this model, a sequence of tokens is processed through multiple layers of multi-head attention-based blocks and feedforward blocks. Within each layer, a local hidden state is computed by summarizing all hidden states, weighted by attention scores. This approach can directly capture the global information of the input sequence. Although initially introduced in the natural language processing field, Transformer-based models have found success in various domains, including computer vision (Dosovitskiy et al., 2021), decision making (Chen et al., 2021), and 3D point cloud processing (Lai et al., 2022).

Recently, Wang et al. (2022) and Dong, Bai, Wei, and Yu (2022) propose to use a Transformer model for 2D affordance learning. However, for flexible manipulation, 6-Degree-of-Freedom (6-DoF) grasping (Sundermeyer, Mousavian, Triebel, & Fox, 2021) that enables a robot arm to pick up objects from arbitrary orientations in a 3D space (Murali, Mousavian, Eppner, Paxton, & Fox, 2020) is needed, which is much more general than 2D grasping (Mahler et al., 2017; Song, Zeng, Lee, & Funkhouser, 2020). Compared to 2D grasping, extending a Transformer model to 6-DoF grasping in a 3D space is a non-trivial problem, which has at least three reasons: (1) Visual observation of a 2D scene can be easily represented as a 2D image, *i.e.* a 2D numerical matrix, while representing a 3D scene has more options, such as point cloud, mesh, voxel, implicit surface, distance function, and so on. (2) Transformer models usually take a 1D sequence as input. How to serialize a 3D representation to a sequence while maintaining the 3D information is a non-trivial problem. (3) The multi-head attention calculation in a Transformer model usually results in heavy computational and memory costs, which is even worse for 3D robotic grasping due to the curse of dimension. How to balance representation learning and the efficiency of a Transformer model is challenging.

In this paper, we propose a novel bio-inspired 6-DoF affordance learning method. In contrast to existing works on affordance learning, our method integrates a Transformer-based model capable of directly encoding a 3D voxelized space. Similar to how primates' brains work, by leveraging this direct encoding, our model exhibits enhanced capability in learning global features, which leads to improved grasping performance. In particular, our approach leverages a Truncated Signed Distance Function (TSDF) as input and utilizes a Transformer model to learn per-voxel feature vectors. These feature vectors are then used to predict grasp parameters for executing a grasp action, including grasp quality, gripper orientation, and gripper width. By employing multi-head attention-based blocks, our method efficiently aggregates global features within each voxel. Specifically, we first serialize the TSDF input as a sequence of feature vectors by dividing it into equally sized 3D grids, and then encode it using a Transformer. This allows the Transformer model to generate a set of aggregated hidden feature vectors through multi-head attention. To obtain per-voxel features for learning grasp parameters, we decode the hidden features of the Transformer model using deconvolution. Furthermore, to integrate global and local features, we utilize skip-connections that connect the global features of the Transformer model with the decoded local features. This merging of global and local features enhances the effectiveness of affordance learning, similar to what primates' brains do. To evaluate the effectiveness of our Transformer-based model for 6-DoF grasp detection, we conducted experiments on a recently proposed pile and packed dataset (Jiang et al. 2021) for a decluttering task. The experimental results demonstrate that our Transformer-based model achieves higher success rates and decluttering rates. Additionally, we conducted an ablation study to analyze the impact of global features, input encoding, and skip-connections on the performance of the proposed model, thereby demonstrating its effectiveness. We deployed our computational model of 6-DoF grasping on a Franka Panda arm to demonstrate its generalization ability. The results highlight the effectiveness of our approach for real-world applications. The contributions of this paper are:

1. Drawing inspiration from cognitive models of grasping, which underscores the importance of global descriptors such as sizes, shapes, and orientations, we present a novel computational model for affordance learning. Leveraging a meticulously designed deep neural network capable of encoding global features directly, our method can achieve efficient and versatile 6-DoF grasping.
2. We propose a novel Transformer-based neural network model that encodes global features effectively through multi-head self-attention. Our model subsequently decodes the global features into local ones via deconvolution and integrates them with skip-connections. This approach effectively learns and integrates global features of a 3D scene with local features, proving to be highly effective for grasping tasks. Additionally, we substantiate the effectiveness of each design element in our model through an ablation study.
3. Our method is evaluated on a recently introduced pile and packed dataset for a decluttering task. The results highlight its superior performance relative to existing 6-DoF robotic grasping methods. Furthermore, we validate our proposed model on a real robot arms, emphasizing its generalization ability and potential for real-world applications.

2. Related work

2.1. Cognitive models of grasping

The early cognitive model of grasping is the FARS (Fagg-Arbib-Rizzolatti-Sakata) model by Fagg and Arbib (1998). In FARS, the visual cortex is divided into two branches: the "what" branch, which flows through the ventral neural pathway, specifically the inferotemporal cortex (IT), controlling object selection, and the "how" branch, which flows through the dorsal neural pathway controlling the execution of grasp actions. In particular, the "how" branch is further divided into two branches: Ventral Intraparietal Area (VIP) for object position and Posterior Intraparietal (PIP) for shape, size, and orientation, all of which serve as global descriptors of an object. PIP then flows to the Anterior Intraparietal Area (AIP), which integrates features from IT and PIP to determine the affordance of an object and generate a set of grasps. The F5 region is responsible for selecting a grasp from AIP for execution. In Cisek (2007), Cisek hypothesizes that multiple grasps generated by the dorsal neural pathway compete with each other, and this competition is influenced by the prefrontal cortex (PFC) and the basal ganglia. Cisek further proposes a computational model to realize this hypothesis. TRoPICALS (Two Route, Prefrontal Instruction, Competition of Affordances, Language Simulation) (Caligiore et al., 2010) considers both FARS and Cisek's models, and further adds linguistic inputs to trigger cognitive simulation of grasping. All of these models emphasize the importance of global descriptors for achieving successful grasps, which serves as inspiration for the work presented in this paper.

Table 1
A summary of typical computational models of robotic grasping.

Research work	Input type	Backbone	Characteristics	Pros and cons
Fang et al. (2020)	Point cloud	PointNet	Learn objectiveness and tolerance scores through PointNet, and sample grasps with these scores	
Wu et al. (2020)	Point cloud	PointNet++	Parameterize a grasp as the two contact points of a gripper, and generate neighbor points to predict a peer	
Wang et al. (2021)	Point cloud	PointNet	Learn the graspness property per-point of a point cloud directly and use it to filter out non-graspable points	
Alliegro et al. (2022)	Point cloud	PointNet++	Use a differentiable sampler to generate grasps and the sampler is trained end-to-end with grasp regression	**Pros**: represent a scene in 3D and can learn 6-DoF grasping parameters easily; **Cons**: Rely on aggregating local features to obtain global features
Varley, DeChant, Richardson, Ruales, and Allen (2017)	Voxelized occupancy	3D-CNN	Complete a point cloud as a voxelized occupancy to improve the performance of grasping	
Liu, Pan, Xu, Ganguly, and Manocha (2019)	Voxelized occupancy	3D-CNN	With an augmented dataset, learn a one-to-one mapping from an input object to its grasp through 3D-CNN	
Breyer et al. (2021)	TSDF	3D-CNN	Take a TSDF as input, encode it with 3D-CNN, and predict grasp quality, gripper orientation, and gripper width	
Jiang, Zhu, Svetlik, Fang, and Zhu (2021)	TSDF	ConvONets (Peng, Niemeyer, Mescheder, Pollefeys, & Geiger, 2020)	Use implicit scene representation and an auxiliary task to improve the performance of grasping	
Wang et al. (2022)	RGB image	Transformer	Use a Transformer model to encode both local and global features of an input image and generate 2D grasps	**Pros**: encode global features directly; **Cons**: cannot execute 6-DoF grasping
Dong et al. (2022)	RGB image	Transformer	Use a Transformer as encoder and a fully convolutional neural network as decoder to improve inductive bias	

2.2. 6-DoF robotic grasping

To perform 6-DoF grasping, the initial step involves acquiring a 3D representation of the scene through a visual perception module, and the 3D representation is then used by computational models to generate grasp parameters, which constitutes the main component of a 6-DoF robotic grasping process. Hence, we begin by providing a concise overview of existing works on stereo vision for grasping. Subsequently, we delve into an extensive review of computational models that are specifically designed for 6-DoF robotic grasping.

2.2.1. Stereo vision for 6-DoF grasping

Existing studies employ various devices to obtain 3D perception for 6-DoF robotic grasping, including stereo cameras (Kakani, Cui, Ma, & Kim, 2021), fish-eye cameras (Kim, Seo, Choi, & Kim, 2016), RGB-D cameras (Fang et al., 2020), and ordinary RGB cameras with multiple views (Breyer et al., 2021). In particular, Kakani et al. (2021) employ a binocular stereo camera to perceive the scene and utilize the obtained information to train a VGG model for regressing contact position and force distribution. This approach has achieved promising results. Kim et al. (2016) use a fisheye camera to guide an aerial manipulator, enabling a wider field of view. The authors have successfully demonstrated the effectiveness of this approach in various manipulation tasks involving large-scale scenes. Fang et al. (2020) apply an RGB-D camera to capture a point cloud of the target scene, and propose a neural network model that generates grasp candidates using this point cloud data. Since 3D depth information can be recovered from multiple views of RGB images (Hartley & Zisserman, 2003), Breyer et al. (2021) propose a method that utilizes a regular camera to perform random multi-view scanning of a scene. This information is then used to reconstruct a 3D voxelized representation of the scene. Subsequently, the authors develop a neural network model that generates grasp parameters based on this 3D representation. This approach does not require dedicated devices and is easy to deploy, which is used in this paper.

2.2.2. Computational models of 6-DoF grasping

Computational models of grasping are primarily employed to implement models for executing grasping tasks. While cognitive models aim to understand the cognitive processes of grasping, computational models focus more on developing models to achieve successful grasping in real-world applications. Therefore, computational models may not always align with cognitive models. For state-of-the-art performance, we only consider deep learning-based methods and give a brief review of 6-DoF robotic grasping according to the input types: unstructured point cloud and structured voxelized space. In Table 1, we provide a summary of the main methods along with their corresponding input types, backbone models, characteristics, pros, and cons. For more thorough reviews of robotic grasping, readers can refer to Newbury et al. (2022), Platt (2023), Zhang, Tang, Sun, and Lan (2022) and the references therein.

6-DoF grasping on point cloud. A point cloud is a set of point coordinates, and each point is possibly associated with some properties, such as RGB colors. Point cloud is an irregular geometric data format, and learning its feature requires permutation invariance of points and transformation invariance of the whole cloud (Qi, Su, et al., 2017). PointNet (Qi, Su, et al., 2017) is the first deep learning model to learn both global and per-point local feature embeddings of a point cloud through multi-layer perceptron and max pooling. PointNet++ (Qi, Yi, et al., 2017) extends PointNet with multiscale learning ability through sampling and grouping. PointNet (Qi, Su, et al., 2017) and PointNet++ (Qi, Yi, et al., 2017) are commonly used as backbone models for

6-DoF robotic grasping. In Fang et al. (2020), Fang et al. use PointNet to encode an input point cloud to obtain the vector representation of each point, and then learn the approaching vectors, operation parameters, and tolerance scores for each point. Subsequentially, the executed grasping points are sampled and filtered by objectiveness and tolerance. Similarly, in Wu et al. (2020), Wu et al. propose Grasp Proposal Networks (GPNet), which uses PointNet++ to encode an input point cloud. Different from Fang et al. (2020), GPNet parameterizes a grasp as the two contact points of the gripper, and generates a set of neighbor points for each point of the cloud to predict its corresponding peer contact point. Due to the unstructured characteristics and the large volume of a point cloud, point cloud-based methods usually need to sample a subset of points as grasp candidates, which results in a performance drop. In Wang et al. (2021), Wang et al. propose to learn *graspness* property per-point and use it to filter out non-graspable points, which is shown to improve grasping success rates significantly on the GraspNet-1Billion dataset. In Alliegro et al. (2022), Alliegro et al. adopt a differentiable sampler and train it end-to-end with grasp regression, which is shown to improve both the performance and running time compared to Wu et al. (2020).

PointNet-based methods mainly rely on local feature learning to aggregate global information progressively, it is still not efficient for robotic grasping, which requires an effective encoding of the global information of an input. In computer vision and graphics, researchers have explored the use of transformer models on point cloud processing (Lu et al., 2022), such as point cloud segmentation (Lai et al., 2022), classification (Yu et al., 2022), and shape completion (Yu et al., 2021). However, the number of points in a point cloud input is not fixed and is too high to be processed with multi-head attention efficiently, which is especially serious in a real robot scenario. How to use transformer models in 6-DoF robotic grasping is still a challenging problem.

6-DoF grasping on voxelized space. Compared to point cloud, voxelized representation partitions a 3D space into uniform grids, which is a structured geometric data format. Commonly used voxelized representation include voxelized occupancy grid (Liu et al., 2019; Varley et al., 2017) and voxelized Truncated Signed Distance Function (TSDF) (Breyer et al., 2021; Jiang et al., 2021). In Varley et al. (2017), Varley et al. use a voxelized occupancy grid to represent objects for shape completion. The authors train a 3D-CNN model and during testing, the model takes an occluded point cloud input and completes the shape for grasp planning. Similarly, in Liu et al. (2019), Liu et al. consider a voxelized occupancy grid input and use 3D-CNN to predict graspable points. They further introduce a consistency loss to ensure a one-to-one mapping from objects to grasp poses. Another option for voxelized 3D space representation is the voxelized Truncated Signed Distance Function (TSDF), where each grid denotes a truncated signed distance to the object surface. In Breyer et al. (2021), Breyer et al. propose Volumetric Grasping Network (VGN) that takes a TSDF as input and uses a 3D-CNN model to learn per-voxel feature vectors, which are then used to regress the grasp quality, gripper orientation, and gripper width for each voxel. However, either voxelized occupancy grid or TSDF requires multi-views to integrate a 3D representation. Jiang et al. (2021) extend VGN by introducing an auxiliary shape completion task, which only requires one view to make grasp predictions. Owing to the use of multi-task learning, the authors also report performance improvement compared with the original VGN.

Existing methods of 6-DoF grasping on voxelized space intensively employ 3D-CNN as backbone models for grasp detection (Platt, 2023). However, convolution operation usually suffers from the local learning problem, which is not efficient to aggregate global features for robotic grasping. Because voxelized space can be viewed as 3D images, it is relatively easy to use transformer models to replace 3D-CNN as the backbone model, which is the focus of this paper.

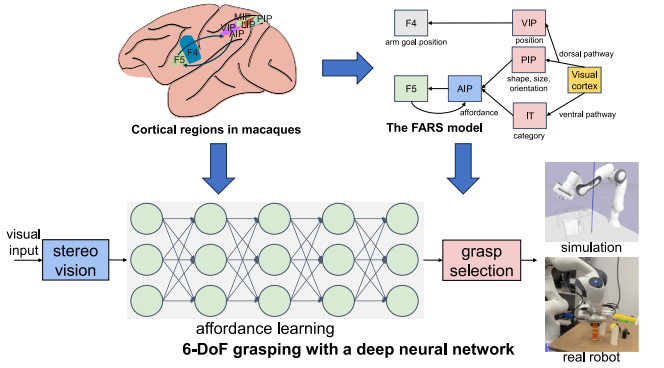

Fig. 1. Overview of our bio-inspired 6-DoF grasping method with a deep neural network.

3. Method

The overview of our bio-inspired 6-DoF grasping method is illustrated in Fig. 1. We draw inspiration from the neural pathway of grasping in cortical regions of macaques and the FARS model (Fagg & Arbib, 1998). Our approach simplifies the grasping pipeline into three modules: stereo vision, affordance learning, and grasp selection. The stereo vision module takes a visual input of the current scene and generates its 3D representation. The affordance learning module takes the 3D scene representation as input and produces graspable points along with their corresponding grasp parameters, referred to as affordances. This module mimics the functionality of AIP in cortical regions of macaques, and is the main focus of this paper. We implement the affordance learning module using a deep neural network, with a specific emphasis on global feature encoding. Using the available grasp candidates, the grasp selection module identifies the most promising grasp, which can be used to control the execution of a simulated or real robot arm. The grasp selection module serves a similar function to F5 in cortical regions of macaques. In the subsequent sections, we provide a more detailed introduction to the framework.

3.1. Stereo vision

The stereo vision module takes the visual input of the current scene as input and generates its corresponding 3D representation. A commonly used dense 3D representation in this module is the Truncated Signed Distance Function (TSDF). TSDF divides the 3D space into a uniform voxel grid, where each grid cell represents the truncated distance to the nearest surface. TSDF is obtained by integrating RGB-D images, which results in a robust representation that is resilient to noise. It provides an implicit representation of the scene, allowing for interpolating to arbitrary scales. This flexibility makes TSDF a versatile choice for capturing detailed spatial information for affordance learning.

We divide the 3D space into a voxel grid of size $N \times N \times N$, where N represents the number of grids in each dimension. Given a voxel grid (i, j, k) in the 3D space, its coordinates in the world frame can be obtained as $p_w = (x_0 + v \cdot i, y_0 + v \cdot j, z_0 + v \cdot k)$, where v is the voxel size and (x_0, y_0, z_0) represents the origin of the 3D space. The coordinates in the camera frame are given by:

$$p_c = R_{wc} p_w + T_c, \tag{1}$$

where R_{wc} is the camera's rotation matrix and T_c is the translation vector. With the intrinsic matrix K of the camera, we can obtain $K p_c = (u, v, d)$, where d represents the depth of this point and (u, v) is its image coordinate. Since a depth camera is used, the surface depth of (u, v) can

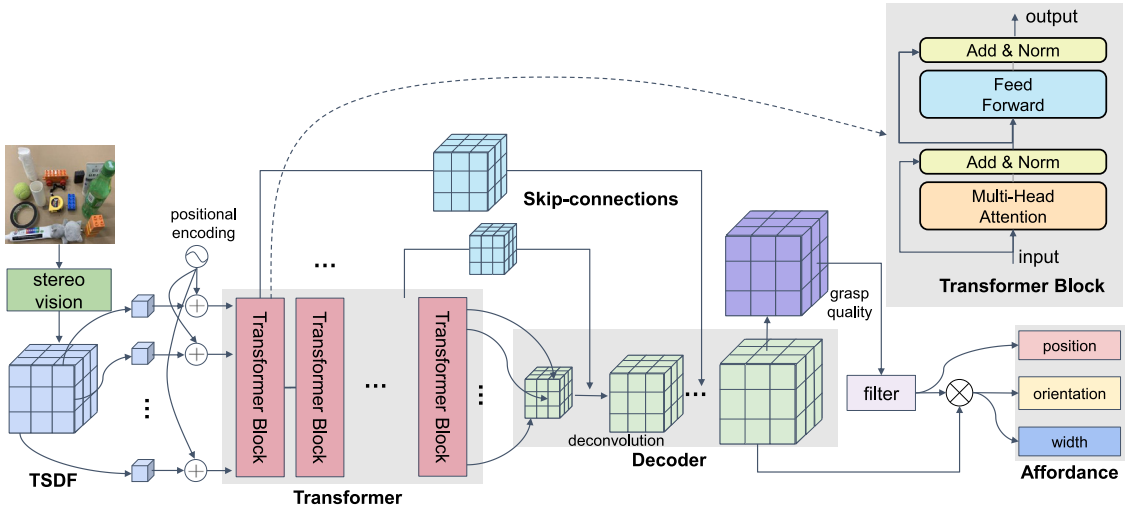

Fig. 2. The architecture of the deep neural network for affordance learning.

be obtained from the camera directly, denoted as d'. The TSDF is then calculated as:

$$\text{TSDF}(i,j,k) = \text{clip}(d' - d), \tag{2}$$

where $\text{clip}(x) = \begin{cases} 1 & x > 1 \\ x & -1 \le x \le 1 \\ -1 & x < -1 \end{cases}$. If multiple depth images are used, the final TSDF value can be obtained by calculating the average of all TSDF values.

3.2. Affordance learning

Utilizing a TSDF as input, we have developed a deep neural network to learn its affordances, which utilizes a Transformer to encode global features effectively. The network architecture is shown in Fig. 2. It begins by serializing an input TSDF into a sequence of tokens. The sequence is then encoded using a deep transformer model, generating multiscale representations for each token. By employing multiple layers of transformer blocks, each token can effectively learn feature vectors that encode global features. To obtain per-voxel feature vectors, we decode the representations through deconvolution. Additionally, we incorporate skip-connections to enhance the merging of global and local features. This approach can effectively encode both global and local features for affordance learning.

3.2.1. Serialization

Transformer models require a sequence of tokens as input. We need to first serialize the TSDF input as a sequence. Moreover, although the attention mechanism can efficiently aggregate global information, it results in heavy computational and memory costs, which is square complexity in terms of the input sequence length. Therefore, we need to limit the cardinality of the input sequence. We treat each non-overlapping cubic patch with size C of the input TSDF as one token and linearly project each token as a 1D vector. In total, we can obtain $M = (\frac{N}{C})^3$ tokens, and adjust the number of tokens by choosing different C values. We then serialize all tokens as $\mathbf{x}' = (x'_0, x'_1, \dots, x'_{M-1}) \in \mathbb{R}^{D' \times M}$, where $x'_i \in \mathbb{R}^{D'}$, D' is the dimension of the projected embedding. $\mathbf{x}'$ is then transformed with a weight matrix W and we add a learnable positional encoding P as:

$$\mathbf{z}^{(0)} = (Wx'_0, Wx'_1, \dots, Wx'_{M-1}) + P, \tag{3}$$

where $W \in \mathbb{R}^{K \times D'}$ and $P \in \mathbb{R}^{K \times M}$. $\mathbf{z}^{(0)}$ is the input token sequence of the transformer model.

3.2.2. Encoding with transformer blocks

The transformer model consists of L transformer blocks and each transformer block transforms an input sequence $\mathbf{z}^{(i)} = (z_0^{(i)}, z_1^{(i)}, \dots, z_{M-1}^{(i)})$ to an output sequence $\mathbf{z}^{(i+1)} = (z_0^{(i+1)}, z_1^{(i+1)}, \dots, z_{M-1}^{(i+1)})$ through a multi-head self-attention block and a feedforward block, where $z_j^{(i)}, z_j^{(i+1)} \in \mathbb{R}^K$.

Multi-head self-attention. Given an input sequence $\mathbf{z}^{(i)} = (z_0^{(i)}, z_1^{(i)}, \dots, z_{M-1}^{(i)})$, Multi-head Self-Attention (MSA) processes it as:

$$\mathbf{z}^{(i+1)} = \text{MSA}(\text{LN}(\mathbf{z}^{(i)})) + \mathbf{z}^{(i)}, \tag{4}$$

where MSA denotes an MSA operation, and LN denotes a layernorm operation. For each hidden state in $z_j^{(i)} \in \mathbf{z}^{(i)}$, where $j \in \{0, 1, \dots, M-1\}$, MSA calculates it as:

$$z_j^{(i+1)} \leftarrow \Big(\text{A}_{W_0^k, W_0^q, W_0^v}(\mathbf{z}^{(i)}, z_j^{(i)}), \dots, \text{A}_{W_{H-1}^k, W_{H-1}^q, W_{H-1}^v}(\mathbf{z}^{(i)}, z_j^{(i)}) \Big) W_o, \tag{5}$$

where W_h^k, W_h^q, W_h^v, W^o are the parameters of MHA, H is the number of heads, $h \in \{0, 1, \dots, H-1\}$, $W_h^k, W_h^q, W_h^v \in \mathbb{R}^{\frac{K}{H} \times K}$, $W^o \in \mathbb{R}^{K \times K}$, and $\text{A}_{W_h^k, W_h^q, W_h^v}(\mathbf{z}^{(i)}, z_j^{(i)})$ is one head transformation, which is calculated as:

$$\text{A}_{W_h^k, W_h^q, W_h^v}(\mathbf{z}^{(i)}, z_j^{(i)}) = \sum_{m=0}^{M-1} \alpha_m W_h^v z_m^{(i)}, \tag{6}$$

where $\alpha_m = \text{softmax}(\frac{(W_h^q z_j^{(i)})^T W_h^k z_m^{(i)}}{\sqrt{K/H}})$.

Feedforward block. The obtained $\mathbf{z}^{(i+1)}$ from multi-head self-attention is then processed with a feedforward block:

$$\mathbf{z}^{(i+1)} \leftarrow \text{MLP}(\text{LN}(\mathbf{z}^{(i+1)})) + \mathbf{z}^{(i+1)}, \tag{7}$$

where MLP denotes a multi-layer perception operation, and LN denotes a layernorm operation.

3.2.3. Decoding with deconvolution and skip-connections

Through multiple layers of transformer blocks, we can obtain hidden representations with different layers, and each layer represents aggregated information at a specific scale. Considering L layers of transformer blocks, we obtain the final layer hidden state as $\mathbf{z}^{L-1} \in \mathbb{R}^{K \times M}$. We project it as a 3D voxelized patch $\mathbf{y}^{(0)} = \text{Proj}(\mathbf{z}^{L-1}) \in \mathbb{R}^{M \times \sqrt[3]{K} \times \sqrt[3]{K} \times \sqrt[3]{K}}$, where Proj denotes the projection operation.

The serialization module projects non-overlapping patches as tokens, which can be viewed as down-sampling the initial TSDF. Therefore, to obtain per-voxel feature vectors of the input TSDF, we up-sample $\mathbf{y}^{(0)}$ with deconvolution. Moreover, to merge multiscale hidden states, we select several layers of the transformer blocks, project them as 3D voxelized patches, up-sample them with deconvolution, and skip-connect them with the decoded state layer by layer:

$$\mathbf{y}^{(i+1)} \leftarrow \text{DECONV}(\mathbf{y}^{(i)}) \oplus \text{DECONV}(\text{Proj}(\mathbf{z}^{l})), \tag{8}$$

where DECONV denotes deconvolution operation and $l \in \{0, 1, \ldots, L-1\}$ is the selected layer. One exception is the final decoding layer $\mathbf{y}^{(L'-1)}$, where we concatenate with the input TSDF processes by convolution and obtain the final output $\mathbf{y}$ as:

$$\mathbf{y} = \text{CONV}\Big(\text{DECONV}(\mathbf{y}^{(L'-1)}) \oplus \text{CONV}(\mathbf{x})\Big), \tag{9}$$

where CONV denotes convolution operation, L' is the number of decoding layer, and $\mathbf{y} \in \mathbb{R}^{D\times N\times N\times N}$ is the feature representation of the input TSDF.

3.2.4. Grasping heads

For each voxel, we predict whether it can be used as a graspable position and its corresponding grasp parameters. Similar to VGN (Breyer et al., 2021), we parameterize a grasp as the position, orientation, and width of a gripper. We add quality $\mathbf{Q} \in [0, 1]^{N\times N\times N}$, gripper orientation $\mathbf{R} \in \mathbb{R}^{4\times N\times N\times N}$, and gripper width $\mathbf{W} \in \mathbb{R}^{N\times N\times N}$ heads through convolution, *i.e.*,

$$\mathbf{Q} = \text{CONV}_1(\mathbf{y}), \mathbf{R} = \text{CONV}_2(\mathbf{y}), \mathbf{W} = \text{CONV}_3(\mathbf{y}). \tag{10}$$

For each voxel, quality q is a score indicating the quality of grasping. A high-quality score indicates a high probability of successful grasping. Orientation $\mathbf{r}$ denotes the orientation of a gripper at a specific voxel to execute grasping, which is parameterized with quaternion, *i.e.* $\mathbf{r} \in \mathbb{R}^4$. Gripper width w_i denotes the gripper width before executing grasping.

3.2.5. Model training

For each voxel, considering a ground-truth grasp $g_i = (q_i, \mathbf{r}_i, w_i)$ and the predicted grasp on this voxel $\hat{g}_i = (\hat{q}_i, \hat{\mathbf{r}}_i, \hat{w}_i)$, where $q_i, \hat{q}_i \in \{0, 1\}$ denote the ground-truth and predicted grasping labels, $\mathbf{r}_i, \hat{\mathbf{r}}_i$ denote the ground-truth and predicted rotation quaternions, and $w_i, \hat{w}_i$ denote the ground-truth and predicted gripper widths, the training objective is calculated as follows (Breyer et al., 2021; Jiang et al., 2021):

$$\mathcal{L}(\hat{g}_i, g_i) = \mathcal{L}_q(\hat{q}_i, q_i) + q_i(\mathcal{L}_r(\hat{r}_i, r_i) + \mathcal{L}_w(\hat{w}_i, w_i)), \tag{11}$$

where $\mathcal{L}_q$ denotes the binary cross-entropy loss, $\mathcal{L}_r = \min(\mathcal{L}_{quat}(\hat{\mathbf{r}}_i, \mathbf{r}_i), \mathcal{L}_{quat}(\hat{\mathbf{r}}_i, \mathbf{r}_{i\pi}))$, $\mathcal{L}_{quat}(\hat{\mathbf{r}}, \mathbf{r}) = 1 - |\hat{\mathbf{r}} \cdot \mathbf{r}|$, $\mathbf{r}_{i\pi}$ denotes the rotation quaternion by rotating $\mathbf{r}_i$ along the gripper's wrist axis with 180°, and $\mathcal{L}_w$ denotes the mean square error. If $q_i = 0$, only the predicted label is considered.

3.3. Grasp selection

The affordance learning network can obtain grasp parameters for each voxel grid, and we need to select the most promising one to execute a grasp action. Generally, selecting a grasp requires considering the external context and specific task (Fagg & Arbib, 1998). For simplicity, we select the grasp with the highest grasp quality $max_{q\in\mathbf{Q}}(q)$. The selected voxel is then used to fit gripper rotation and width. Considering the voxel size v and the rigid transformation T between a simulated or real robot platform and the TSDF volume, the robot gripper configuration for executing a grasp action can be calculated as:

$$p_g = T\frac{(i, j, k)^T}{v}, \mathbf{r}_g = T\mathbf{r}, w_g = \frac{w}{v} \tag{12}$$

where (i, j, k) denotes the indices of a graspable voxel, p_g denotes the position of the gripper center, $\mathbf{r}_g$ denotes the orientation of the gripper, and w_g denotes the width of the gripper. $p_g, \mathbf{r}_g, w_g$ are in the base frame.

Table 2
Statistics of the pile and packed dataset.

	#scenes	#grasps	#test objects
Pile	83 305	1 564 546	40
Packed	16 640	638 012	16

4. Experiment

To verify the effectiveness of the proposed method, we conducted a set of simulation experiments following the setting in Breyer et al. (2021) and Jiang et al. (2021). Additionally, to demonstrate its potential in real-world applications, we deployed the proposed model on a physical robotic arm, *i.e.* the Franka Panda[2], and conducted another comparison experiment. The demonstrations on a real hardware platform can further verify the generalization ability of our method.

4.1. Pile and packed dataset

We adopt the dataset used in Jiang et al. (2021), which adopts the same data generation procedure as in Breyer et al. (2021). We term it as a pile and packed dataset. The pile and packed dataset contains two scenarios for a decluttering task, which requires generating a set of grasps that can move a clutter of objects to a target bin. Each scenario is constructed from 303 training objects following the procedure in Breyer et al. (2021). The pile scenario is a fixed-size workspace containing objects dropped randomly. The packed scenario is a fixed-size workspace containing objects placed randomly at their canonical pose. The statistics of the pile and packed dataset is shown in Table 2. For each scenario, we use 90% of the grasps as training data and the remaining 10% of as validation data. For testing, there are 40 objects that do not appear in the 303 training objects, which are used to compose a simulation environment on the fly. The pile scenario uses all 40 objects, while the packed scenario requires objects to have proper sizes, and only uses 16 objects among them.

4.2. Baselines

Similar to Jiang et al. (2021), we adopt SHAF, GPN, and VGN as baselines. SHAF uses a heuristic to select objects with the highest point to grasp. GPN (Gualtieri, ten Pas, Saenko, & Platt, 2016) is a classical hypothesizing and testing method (Platt, 2023) that first samples grasping poses in neighborhoods of each point in an input point cloud, and then classifies whether the sampled grasp works or not. VGN (Breyer et al., 2021) uses 3D convolution to learn local features of a 3D scene, and obtains global features through aggregating several 3D convolution layers, which is the main baseline that we target to compare with.

4.3. Metrics

We adopt grasp success rates (GSR) and declutter rates (DR) as evaluation metrics. GSR is calculated as the ratio between the number of successful grasping n_{suc} and the number of grasps predicted n_{pred}: $GSR = \frac{n_{\text{suc}}}{n_{\text{pred}}}$. DR is calculated as the ratio between the number of removed objects n_{rem} and the total number of objects n_{obj} in the current scene: $DR = \frac{n_{\text{rem}}}{n_{\text{obj}}}$.

4.4. Implementation

We implement our affordance learning neural network in PyTorch[3]. With the automatic differentiation engine implemented in PyTorch,

[2] https://www.franka.de
[3] https://pytorch.org.

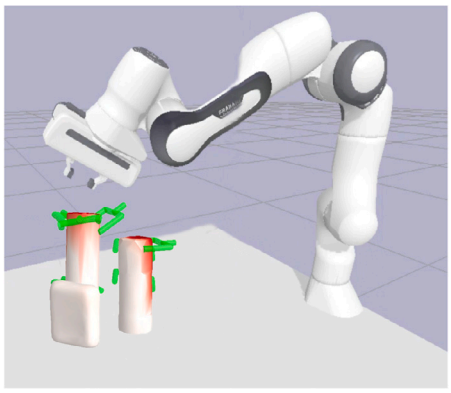

Fig. 3. A snapshot of the simulation environment.

Table 3
The comparison of simulation results on packed and pile scenarios.

Method	Packed		Pile	
	GSR(%)	DR(%)	GSR(%)	DR(%)
SHAF	56.6 ± 2.0	58.0 ± 3.0	50.7 ± 1.7	42.6 ± 2.8
GPD	35.4 ± 1.9	30.7 ± 2.0	17.7 ± 2.3	9.2 ± 1.3
VGN	74.5 ± 1.3	79.2 ± 2.3	60.7 ± 4.2	44.0 ± 4.9
VGN-R	74.4 ± 3.6	79.8 ± 3.1	62.5 ± 2.4	46.4 ± 2.9
Ours	**79.7** ± 2.0	**85.9** ± 1.7	**66.9** ± 3.0	**50.9** ± 3.5

we can train the network with the chain-rule automatically without explicitly calculating gradients. The physical engine of the simulation environment is PyBullet[4], and we used the default Franka Panda arm in PyBullet for grasping experiments, as shown in figure 3. For fair comparison, the input TSDF is divided into 40 × 40 × 40 grids, *i.e.* $N = 40$. To save GPU memory and computational time, we choose a patch size $C = 8$, which results in $M = 5 \times 5 \times 5 = 125$ tokens. Similar to Jiang et al. (2021), we train the two scenarios pile and packed separately, and use the same setting in the remaining experiments. We use a transformer model with $L = 12$ layers to encode the input TSDF, and decode it with $L' = 3$ deconvolution blocks. Empirically, we choose the convoluted input TSDF and the outputs of layer 4, 7 of the transformer model and skip-connect them to the three deconvolution blocks sequentially. Adam optimizer is used with the learning rate 0.0001 and batch size 64. We set $K = 768, H = 12, D = 16$. The model is trained in 21 epochs and we use the checkpoint of the last epoch for evaluation. For training the models, we use a server with Intel(R) Xeon(R) Silver 4210R CPU @ 2.40 GHz, 64 GB memory, and NVIVIDA GTX 3090 GPU (×1). The training time lasts about 60 h for each scenario.

4.5. Results and analysis

Similar to Jiang et al. (2021), for testing each scenario, we ran 100 simulation rounds and repeated five times with fixed seeds. We report the mean and standard deviation of the results. For SHAF, GPD, we take the results from Jiang et al. (2021). For VGN, we take the results from Jiang et al. (2021) (denoted as VGN) and also evaluate the results in the same setting with the pre-trained models released in Jiang et al. (2021) (denoted as VGN-R). The simulation results are shown in Table 3.

Based on the results, our method demonstrates a significant performance advantage over SHAF, GPD, and VGN in both grasp success rates (GSR) and declutter rates (DR) for both scenarios. The utilization of a transformer model enables our method to effectively aggregate global information from visual observations, resulting in an approximate 5% improvement in both GSR and DR. This finding aligns with the cognitive models of grasping, where global information plays a critical role in affordance learning. By incorporating a transformer model to encode TSDF global features and integrating them with decoded local features through skip-connections, we can leverage this biological fact more effectively. Furthermore, this approach can be easily generalized and transferred to other grasp learning methods. Another notable finding is that the pile scenario is more challenging than the packed scenario, as evidenced by the performance decline across all methods. Still, our method still achieves the best results. The pile scenario represents a fixed-size workspace where objects are randomly dropped. To provide further insights, we conduct a detailed analysis of the underlying reasons in the following failure case section.

We further analyzed the attention maps of our affordance learning neural network to understand how the Transformer encodes global features. Additionally, to assess the feasibility of applying the proposed method in real-world applications, we evaluated the running time, memory usage, and generalization ability.

Table 4
The comparison of the average time for generating all grasps of one input TSDF in milliseconds (ms).

	Packed	Pile
VGN-R	11.0 ± 0.5	11.2 ± 0.6
Ours	24.3 ± 0.7	24.6 ± 1.1

4.5.1. Global feature encoding

To elucidate how our Transformer-based model encodes global features, we examined the attention maps of the Transformer model. Fig. 4 illustrates an example where we designed a random scene with three objects. Owing to space constraints, we only depict the last six layers, and each with all 12 attention heads. Generally, we observe that each voxel grid concentrates on one or two specific voxel grid, which varies across different heads and layers. This is analogous to how humans process global information. For instance, when calculating the volume of a cube, we focus on a specific vertex and multiply the height, width, and length to achieve the volume value. Thanks to the use of multiple heads and layers, our model can focus on a sufficient number of relevant positions. Furthermore, the application of multi-head attention allows us to attend to any positions of interest and efficiently aggregate global information.

4.5.2. Running time

Since robotic grasping is a fundamental task of robot manipulation, it is critical to generate successful grasps in real time. With the same experiment setting, we calculated the average time of generating all grasps for one input TSDF. The results are shown in Table 4. Because the computation process of a transformer model is relatively heavy, our method nearly doubles the running time of VGN. However, it is still in a reasonable range for real-time usage. Moreover, since transformers gradually become universal models for machine learning, dedicated devices are available (Wang et al., 2020) for further improving the performance.

4.5.3. Memory usage

Robotic arms typically operate using embedded systems, which often have limited memory capacity. Therefore, the memory usage of a method becomes a crucial consideration. Since our models are designed to run on GPU, we conducted experiments with the same settings and calculated the GPU memory usage for generating all grasps for a single input TSDF. Regardless pile and packed scenario, our model uses 3067 MB GPU memory, while VGN-R uses 1629 MB GPU memory. Our method employ a Transformer model, which is significantly larger than

[4] https://pybullet.org.

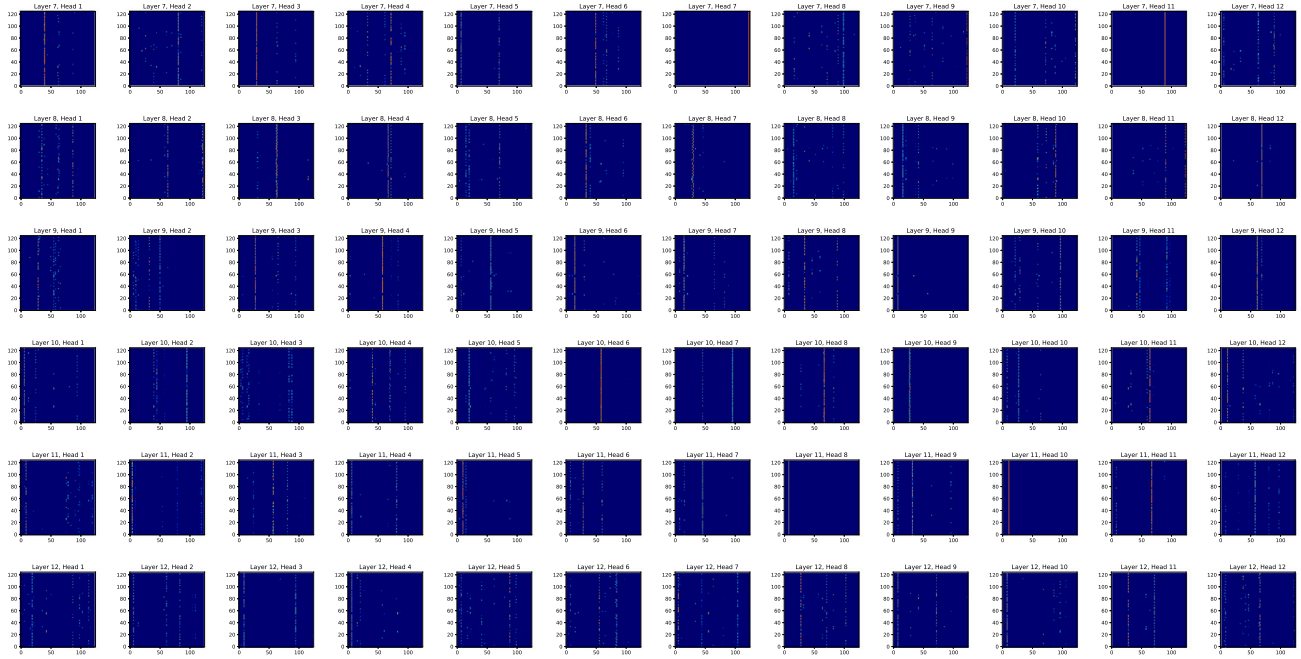

Fig. 4. An example of the attention map learned by our Transformer-based affordance learning neural network.

Table 5
The comparison results of generalization performance on new scenarios.

Method	Pile→Packed		Packed→Pile	
	GSR(%)	DR(%)	GSR(%)	DR(%)
VGN-R	60.1 ± 1.8	63.2 ± 1.8	44.9 ± 1.2	33.5 ± 1.3
Ours	**74.0** ± 2.8	**78.9** ± 2.7	**49.5** ± 1.4	**39.8** ± 1.7

Table 6
Results of our ablation study on global features, input encoding, and skip-connections.

Method	Packed		Pile	
	GSR(%)	DR(%)	GSR(%)	DR(%)
no-attn	71.3 ± 3.4	77.9 ± 3.3	63.3 ± 3.1	49.6 ± 3.8
8 tokens	68.7 ± 2.8	75.5 ± 2.3	50.6 ± 2.2	38.5 ± 3.8
64 tokens	75.0 ± 2.2	82.6 ± 1.9	56.4 ± 3.3	43.0 ± 4.0
no-skip	76.6 ± 1.7	83.1 ± 2.3	66.4 ± 1.9	50.5 ± 2.8
Ours	**79.7** ± 2.0	**85.9** ± 1.7	**66.9** ± 3.0	**50.9** ± 3.5

the VGN model. However, the memory requirements remain within acceptable limits for modern industrial computation devices[5].

4.5.4. Generalization on new scenarios

Another important aspect for real-world usage of robotic grasping is the generalization ability of a model on new scenarios. With the same experiment setting, we tested the performance of the model on the packed scenario, while the model is trained with the pile data (denoted as Pile→Packed), and the performance of the model on the pile scenario, while the model is trained with the packed data (denoted as Packed→Pile). The results are shown in Table 5. In general, the performance dropped for both VGN and our method. However, it is interesting to find that our method still outperforms VGN significantly, especially for the packed scenario, where the GSR and DR of our method are higher than the ones of VGN for about 10%. This indicates that our method can generalize better than VGN on new scenarios, which is more suitable for real usage.

4.6. Ablation study

To gain deeper insights into the design of our model, we conducted an ablation study. In the same experimental setting, we examined the influence of global features, input encoding, and skip-connections.

To investigate the impact of global features, we applied a diagonal identity matrix to mask the attention in the Transformer encoder, which allows each hidden vector to rely solely on its local feature (referred to as **no-attn**). The result is summarized in Table 6. It is evident from the result that global features play a critical role in the performance of our Transformer model. Specifically, **no-attn** exhibits an approximately 8% drop in performance for the packed scenario and a 3% drop for the pile scenario. This indicates that encoding global features directly can significantly enhance the performance of affordance learning.

For input encoding, we serialized the input TSDF with different patch sizes, specifically 20 and 10. This results in a variation in the number of tokens, namely 8 tokens and 64 tokens. The different encoding schemes may lead to varying performance outcomes. The summarized results can be found in Table 6. Our method utilizes the default patch size of 8, resulting in 125 tokens. From the table, we can find that reducing the number of tokens significantly decreases performance. For instance, in both the packed and pile scenarios, using 8 tokens leads to a performance drop of approximately 10% compared to the case of using 125 tokens. On the other hand, although increasing the token number can improve performance, it comes at the expense of increased running time and memory usage. Thus, finding the right balance between input encoding and performance is crucial for real-world applications.

Furthermore, we conducted experiments to evaluate the impact of skip-connections by testing a configuration where they were removed, referred to as **no-skip**. In this setup, the decoder does not directly integrate higher-layer global features with lower-layer features, and global features are decoded layer-by-layer. The results of this ablation study can be found in Table 6. The **no-skip** configuration exhibits a performance drop of approximately 3% for the pile scenario and 0.5% for the packed scenario. The performance improvement achieved through skip-connections was not substantial, as we employed them empirically. To further explore this aspect, it is necessary to conduct

[5] https://www.nvidia.com/en-us/autonomous-machines/embedded-systems.

Fig. 5. A snapshot of a Franka Panda arm with a parallel-jaw gripper used in our real-world experiments.

Table 7
The comparison of experimental results in real-world packed and pile scenarios with a Franka Panda arm.

Method	Packed		Pile	
	GSR(%)	DR(%)	GSR(%)	DR(%)
VGN	72.9	71.7	69.2	60.0
Ours	**79.0**	**81.7**	**70.5**	**71.7**

additional research on automatic architecture discovery methods. This will help us gain a deeper understanding of the potential benefits and optimal utilization of skip-connections in our model.

4.7. Real robot experiments

In order to evaluate the grasping performance of the proposed model in real-world scenarios, we conducted a decluttering experiment using a 7-DoF Franka Panda robotic arm. As shown in Fig. 5, the end-effector of the Franka Panda arm is a parallel-jaw gripper, which is identical to the simulation experiment setup. Using a physical robotic arm allows us to better assess the generalization capabilities in real-world deployment of the proposed model.

In the real grasping experiments, we utilized 13 everyday objects, as shown in Fig. 6. As VGN is the main baseline that we target to compare with and it outperforms SHAF and GPN in the simulation, we only compared VGN in the real robot experiment. Consistent with the simulation experiment, we tested VGN and our method in two scenarios: packed and piled. We performed 10 rounds of decluttering experiments for each scenario, employing different robotic arms. In each round, we randomly selected six objects from the test set and placed them on a flat surface. For each input scene, the method can generate a set of graspable candidates. We chose the grasp that has the highest grasping point to avoid colliding with other objects during grasping.

If the robotic arm can grasp the object and places it in a storage bin next to the workspace, we record it as a successful grasp. We repeated the candidate generation and execution loop until one of the following three conditions is met:

- (1) All the objects on the flat surface are cleared;
- (2) The model cannot generate any grasp candidates;
- (3) Two consecutive grasp actions fail to grasp and move the target object to the storage bin.

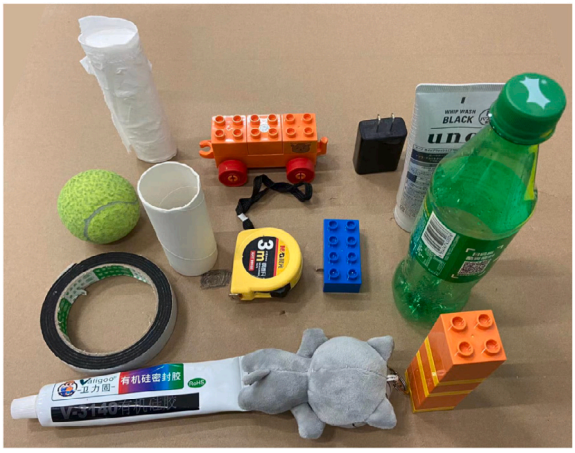

Fig. 6. 13 objects used in the grasping experiment.

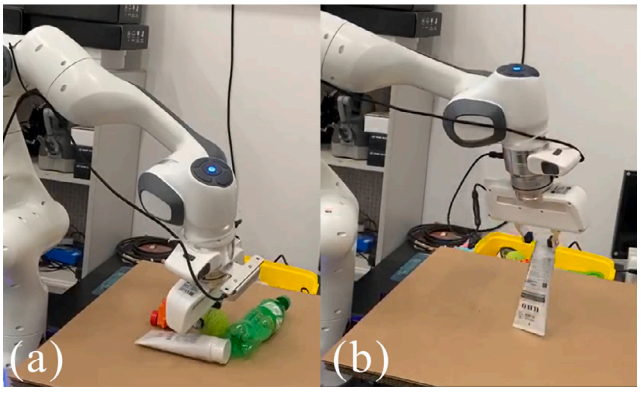

Fig. 7. Typical failure cases: (a) the collision between the robotic arm and the object occurred, (b) the gripper fingers slipped off the object due to the small contact surface.

The grasping experiment results are shown in Table 7. Overall, our method achieved higher success rates and cleared more objects, in line with the simulation results. Since the experiment setting with the Franka Panda arm was the same as the simulation experiment, we observed a similar performance improvement with our method. This result aligns with previous research works (Breyer et al., 2021; Jiang et al., 2021), which suggests that simulation results can indicate the performance of a model in real robot grasping tasks.

Failure cases. To investigate the reasons behind the failure cases of our transformer-based model in particular scenarios, we video-recorded the entire experiment and analyzed the failure cases one by one. In general, we observed that the failure cases could be categorized into two categories: (1) collisions between the robotic arm and the object during the grasping execution; (2) insufficient contact surface between the end-effector and the target object, as shown in Fig. 7. The first case occurred when the robotic arm collided with the object during the grasping process. In that case, if the end-effector does not have sufficient friction force, the grasped object is easy to slip out of the gripper. In the second case, the gripper was unable to establish a stable and secure grasp due to the limited contact area. This issue becomes particularly critical in the pile scenario, where objects tend to overlap with each other.

The potential reasons for these failures are that (1) the grasping process does not plan a collision-free path explicitly and (2) the simulation environment does not fully replicate the real world. It is relatively easy to strengthen the system with off-the-shelf path planning modules. However, the gap between simulation and real-world scenarios is still a crucial problem for 6-DoF grasping research. The data generation

procedure used in previous works (Breyer et al., 2021; Jiang et al., 2021) does not consider abnormal cases that may occur in real-world scenarios.

5. Limitations and future work

The current work has several limitations that provide potential opportunities for future research. Firstly, we solely focus on a fixed-size TSDF, and there is potential for improvement by exploring implicit representation methods. Additionally, investigating the integration of fisheye cameras as the stereo vision module and leveraging them for large scene grasping could be an interesting direction for further exploration. Secondly, our current work only emphasizes the backbone computational model. In future studies, we intend to integrate stereo vision techniques, such as fisheye cameras, along with motion planning and collision avoidance systems. This integration would enhance the practicality of the system for real-world applications. Thirdly, some parameters of our current model, such as skip-connections, were configured empirically, as demonstrated in the ablation study. To improve performance, we plan to employ neural architecture search methods to automatically search for optimal configurations. Lastly, the current approach only attempts a single grasp, and if the initially predicted affordance is incorrect, the grasp fails. Exploring the emulation of the human approach to interactive grasping presents an intriguing opportunity to enhance grasping performance. In this method, we adopt a feedback-driven approach that allows for iterative refinement of our grasp predictions. If our initial affordance prediction is incorrect and the object is not successfully grasped on the first attempt, we initiate a replanning phase. During this phase, we leverage the feedback received from the failed grasp to make a new affordance prediction. We then iteratively interact with the object, continually adjusting our grasping strategy based on the acquired feedback, a successful grasp is achieved.

6. Conclusion

This paper considers a 6-DoF affordance learning problem, and we propose a novel bio-inspired transformer-based model to learn per-voxel features to regress grasp parameters. The model first serializes an input TSDF as a sequence of tokens and then encodes the input sequence with a stack of transformer blocks. The output of the transformer model is used to decode grasp parameters that are needed for a robot arm to execute a grasp action. Thanks to the use of a transformer model, we can learn global features of a scene effectively, similar to what primates' brains do. With the help of multiple skip-connections, we can elegantly integrate global features with local ones to generate grasp candidates. Through intensive experiments, we demonstrate that our method can effectively learn affordance for 6-DoF robotic grasping tasks. Additionally, we deployed the proposed model on a real robot arm, *i.e.* a Franka Panda with a parallel-jaw gripper. A comparison experiment shows that our model is able to generate stable grasp candidates, indicating its potential for real-world applications.

CRediT authorship contribution statement

Zhenjie Zhao: Conceptualization, Methodology, Software, Writing, Funding acquisition, Original draft, Writing – review & editing. **Hang Yu:** Methodology, Real Robot Experiment, Writing – review & editing. **Hang Wu:** Methodology, Writing – review & editing. **Xuebo Zhang:** Methodology, Writing, Supervision, Funding acquisition, Review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Code availability

Code will be made available on request.

Acknowledgments

This work is sponsored by National Natural Science Foundation of China under grant 62106109, and in part by K. C. Wong Education Foundation, Hong Kong.

References

Alliegro, A., Rudorfer, M., Frattin, F., Leonardis, A., & Tommasi, T. (2022). End-to-end learning to grasp via sampling from object point clouds. *IEEE Robotics and Automation Letters*, *7*(4), 9865–9872. http://dx.doi.org/10.1109/LRA.2022.3191183.

Breyer, M., Chung, J. J., Ott, L., Siegwart, R., & Nieto, J. (2021). Volumetric Grasping Network: Real-time 6-DoF grasp detection in clutter. In *Proceedings of machine learning research: vol. 155, Proceedings of the 2020 conference on robot learning* (pp. 1602–1611).

Caligiore, D., Borghi, A. M., Parisi, D., & Baldassarre, G. (2010). TRoPICALS: A computational embodied neuroscience model of compatibility effects. *Psychological Review*, *117*(4), 1188.

Chen, L., Lu, K., Rajeswaran, A., Lee, K., Grover, A., Laskin, M., et al. (2021). Decision transformer: Reinforcement learning via sequence modeling. In *Advances in neural information processing systems: vol. 34*, (pp. 15084–15097).

Cisek, P. (2007). Cortical mechanisms of action selection: The affordance competition hypothesis. *Philosophical Transactions of the Royal Society, Series B (Biological Sciences)*, *362*(1485), 1585–1599.

Cong, Y., Chen, R., Ma, B., Liu, H., Hou, D., & Yang, C. (2021). A comprehensive study of 3-D vision-based robot manipulation. *IEEE Transactions on Cybernetics*, *53*(3), 1–17. http://dx.doi.org/10.1109/TCYB.2021.3108165.

Detry, R., Kraft, D., Kroemer, O., Bodenhagen, L., Peters, J., Krüger, N., et al. (2011). Learning grasp affordance densities. *Paladyn*, *2*, 1–17.

Di Natali, C., Buzzi, J., Garbin, N., Beccani, M., & Valdastri, P. (2015). Closed-loop control of local magnetic actuation for robotic surgical instruments. *IEEE Transactions on Robotics*, *31*(1), 143–156. http://dx.doi.org/10.1109/TRO.2014.2382851.

Dong, M., Bai, Y., Wei, S., & Yu, X. (2022). Robotic grasp detection based on transformer. In *Intelligent robotics and applications: 15th international conference, ICIRA 2022, Harbin, China, August 1–3, 2022, proceedings, Part IV* (pp. 437–448). Berlin, Heidelberg: Springer-Verlag, http://dx.doi.org/10.1007/978-3-031-13841-6_40.

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., et al. (2021). An image is worth 16 × 16 words: Transformers for image recognition at scale. In *International conference on learning representations*.

Fagg, A. H., & Arbib, M. A. (1998). Modeling parietal–premotor interactions in primate control of grasping. *Neural Networks*, *11*(7–8), 1277–1303.

Fang, H.-S., Wang, C., Gou, M., & Lu, C. (2020). GraspNet-1Billion: A large-scale benchmark for general object grasping. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*.

Gualtieri, M., ten Pas, A., Saenko, K., & Platt, R. (2016). High precision grasp pose detection in dense clutter. In *2016 IEEE/RSJ international conference on intelligent robots and systems* (pp. 598–605). http://dx.doi.org/10.1109/IROS.2016.7759114.

Hartley, R., & Zisserman, A. (2003). *Multiple view geometry in computer vision* (2nd ed.). USA: Cambridge University Press.

Huang, B., Han, S. D., Boularias, A., & Yu, J. (2021). DIPN: Deep interaction prediction network with application to clutter removal. In *2021 IEEE international conference on robotics and automation* (pp. 4694–4701). http://dx.doi.org/10.1109/ICRA48506.2021.9561073.

Jiang, Z., Zhu, Y., Svetlik, M., Fang, K., & Zhu, Y. (2021). Synergies between affordance and geometry: 6-DoF grasp detection via implicit representations. In *Robotics: science and systems (RSS)*.

Kakani, V., Cui, X., Ma, M., & Kim, H. (2021). Vision-based tactile sensor mechanism for the estimation of contact position and force distribution using deep learning. *Sensors*, *21*(5), http://dx.doi.org/10.3390/s21051920, URL https://www.mdpi.com/1424-8220/21/5/1920.

Kim, S., Seo, H., Choi, S., & Kim, H. J. (2016). Vision-guided aerial manipulation using a multirotor with a robotic arm. *IEEE/ASME Transactions on Mechatronics*, *21*(4), 1912–1923. http://dx.doi.org/10.1109/TMECH.2016.2523602.

Kopuklu, O., Kose, N., Gunduz, A., & Rigoll, G. (2019). Resource efficient 3D convolutional neural networks. In *Proceedings of the IEEE/CVF international conference on computer vision (ICCV) workshops*.

Lai, X., Liu, J., Jiang, L., Wang, L., Zhao, H., Liu, S., et al. (2022). Stratified transformer for 3D point cloud segmentation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 8500–8509).

Liu, M., Pan, Z., Xu, K., Ganguly, K., & Manocha, D. (2019). Generating grasp poses for a high-DoF gripper using neural networks. In *2019 IEEE/RSJ international conference on intelligent robots and systems* (pp. 1518–1525). http://dx.doi.org/10.1109/IROS40897.2019.8968115.

Lu, D., Xie, Q., Wei, M., Gao, K., Xu, L., & Li, J. (2022). Transformers in 3D point clouds: A survey. arXiv preprint arXiv:2205.07417.

Mahler, J., Liang, J., Niyaz, S., Laskey, M., Doan, R., Liu, X., et al. (2017). Dex-Net 2.0: Deep learning to plan robust grasps with synthetic point clouds and analytic grasp metrics. In *Robotics: science and systems (RSS)*.

Murali, A., Mousavian, A., Eppner, C., Paxton, C., & Fox, D. (2020). 6-DoF grasping for target-driven object manipulation in clutter. In *2020 IEEE international conference on robotics and automation* (pp. 6232–6238). IEEE.

Newbury, R., Gu, M., Chumbley, L., Mousavian, A., Eppner, C., Leitner, J., et al. (2022). Deep learning approaches to grasp synthesis: A review. arXiv preprint arXiv:2207.02556.

Oliver, G., Gil, P., & Torres, F. (2020). Robotic workcell for sole grasping in footwear manufacturing. In *2020 25th IEEE international conference on emerging technologies and factory automation, vol. 1* (ETFA), (pp. 704–710). IEEE.

Peng, S., Niemeyer, M., Mescheder, L., Pollefeys, M., & Geiger, A. (2020). Convolutional occupancy networks. In *Computer Vision - ECCV 2020: 16th European conference, Glasgow, UK, August 23-28, 2020, proceedings, Part III* (pp. 523–540). Berlin, Heidelberg: Springer-Verlag, http://dx.doi.org/10.1007/978-3-030-58580-8_31.

Platt, R. (2023). Grasp learning: Models, methods, and performance. *Annual Review of Control, Robotics, and Autonomous Systems*, *6*(1), http://dx.doi.org/10.1146/annurev-control-062122-025215.

Qi, C. R., Su, H., Mo, K., & Guibas, L. J. (2017). PointNet: Deep learning on point sets for 3D classification and segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*.

Qi, C. R., Yi, L., Su, H., & Guibas, L. J. (2017). PointNet++: Deep hierarchical feature learning on point sets in a metric space. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, & R. Garnett (Eds.), *vol. 30*, *Advances in neural information processing systems*. Curran Associates, Inc..

Song, M., & Xin, S. (2021). Robot autonomous sorting system for intelligent logistics. In *2021 IEEE international conference on computer science, electronic information engineering and intelligent control technology* (pp. 80–83). IEEE.

Song, S., Zeng, A., Lee, J., & Funkhouser, T. (2020). Grasping in the wild: Learning 6-DoF closed-loop grasping from low-cost demonstrations. *IEEE Robotics and Automation Letters*, *5*(3), 4978–4985.

Sundermeyer, M., Mousavian, A., Triebel, R., & Fox, D. (2021). Contact-GraspNet: Efficient 6-dof grasp generation in cluttered scenes. In *2021 IEEE international conference on robotics and automation* (pp. 13438–13444). http://dx.doi.org/10.1109/ICRA48506.2021.9561877.

Varley, J., DeChant, C., Richardson, A., Ruales, J., & Allen, P. (2017). Shape completion enabled robotic grasping. In *2017 IEEE/RSJ international conference on intelligent robots and systems* (pp. 2442–2447). http://dx.doi.org/10.1109/IROS.2017.8206060.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., et al. (2017). Attention is all you need. In *Advances in neural information processing systems*: *vol. 30*.

Wang, C., Fang, H.-S., Gou, M., Fang, H., Gao, J., & Lu, C. (2021). Graspness discovery in clutters for fast and accurate grasp detection. In *2021 IEEE/CVF international conference on computer vision* (pp. 15944–15953). http://dx.doi.org/10.1109/ICCV48922.2021.01566.

Wang, H., Wu, Z., Liu, Z., Cai, H., Zhu, L., Gan, C., et al. (2020). HAT: Hardware-aware transformers for efficient natural language processing. In *Annual conference of the association for computational linguistics*.

Wang, S., Zhou, Z., & Kan, Z. (2022). When transformer meets robotic grasping: Exploits context for efficient grasp detection. *IEEE Robotics and Automation Letters*, *7*(3), 8170–8177. http://dx.doi.org/10.1109/LRA.2022.3187261.

Wu, C., Chen, J., Cao, Q., Zhang, J., Tai, Y., Sun, L., et al. (2020). Grasp Proposal Networks: An end-to-end solution for visual learning of robotic grasps. In *Advances in neural information processing systems*: *vol. 33*, (pp. 13174–13184).

Yu, X., Rao, Y., Wang, Z., Liu, Z., Lu, J., & Zhou, J. (2021). PoinTr: Diverse point cloud completion with geometry-aware transformers. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 12498–12507).

Yu, X., Tang, L., Rao, Y., Huang, T., Zhou, J., & Lu, J. (2022). Point-BERT: Pre-training 3D point cloud transformers with masked point modeling. In *2022 IEEE/CVF conference on computer vision and pattern recognition* (pp. 19291–19300). http://dx.doi.org/10.1109/CVPR52688.2022.01871.

Zhang, H., Tang, J., Sun, S., & Lan, X. (2022). Robotic grasping from classical to modern: A survey. arXiv preprint arXiv:2202.03631.